

BOOK A DOCTOR

A Premium Full-Stack MERN Healthcare Application



System Architecture & Design Specifications

DEVELOPED BY: Antigravity AI Codebase Builder
DATE: July 2026
VERSION: 1.0.0 (Production-Ready)

1. Executive Summary & Table of Contents

This documentation serves as the blueprint for the "Book a Doctor" medical coordination platform, detailing the databases, routers, front-end views, and configurations. It compiles full specifications of schemas and API architectures designed to support concurrent scheduling loads.

Table of Contents

1. Executive Summary & TOC	2
2. MERN System Architecture	3
3. Database Design & Models (User, Doctor, Appointment, Admin)	4
4. Authentication & Security Flow (JWT, Bcrypt)	6
5. Frontend Layout & State (Vite React, Context, Axios)	7
6. Local Development Setup & Verification Checklist	8

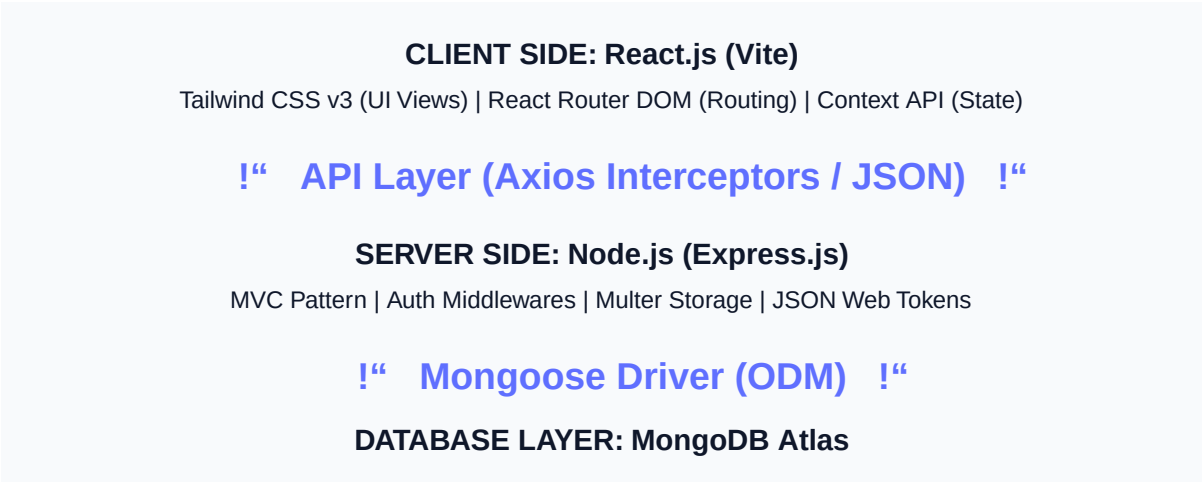
Project Goals & Key Features

- Role-based dashboard queues supporting Patient profiles, Doctor schedules, and Admins.
- Encrypted token authentication (JWT) restricting API access to validated clients.
- Dynamically computed slot allocations checking clinic schedules and time zone overlaps.
- Cloud-supported assets handling (Multer + Cloudinary) for medical files and avatar uploads.

2. MERN System Architecture

The Book a Doctor platform is architected around a Decoupled Client-Server Paradigm, offering strict separation between frontend viewstates and backend business logic. This layout allows independent scaling, deployment, and auditing.

Architecture Diagram Representation



Key Architectural Components

Model-View-Controller (MVC) Pattern

The backend separates MongoDB Mongoose models, routing pathways, controller operations, and security middlewares. This ensures cleaner unit test audits and simplifies route expansion.

Unified Authentication Gateway

A custom middleware checks for Bearer header tokens, decrypts client data across three collections (Admin, Doctor, Patient), and loads context parameters before calling next routes.

Vite React Framework

Compiles bundles in seconds using esbuild. Built with optimized chunks routing, minimizing browser load latency.

3. Database Design & Models

The application defines four separate database collections inside MongoDB using Mongoose schemas. This decoupling removes large populated joins, improves query speeds, and enables custom validation rules.

Schema 1: User (Patients)

Stores credentials and contact data for patient accounts. Implements automatic bcrypt hashing on save.

```
• name:           { Type: String, Required: true }
• email:          { Type: String, Required: true, Unique: true }
• password:       { Type: String, Required: true, Select: false }
• phone:          { Type: String, Required: true }
• profileImage:   { Type: String, Default: "blank-profile-url" }
• role:           { Type: String, Enum: ["patient", "admin"], Default: "patient" }
• timestamps:     { createdAt, updatedAt }
```

Schema 2: Admin

Dedicated schema for administrators. Excluded from patient listing functions.

```
• name:           { Type: String, Required: true }
• email:          { Type: String, Required: true, Unique: true }
• password:       { Type: String, Required: true }
• role:           { Type: String, Default: "admin" }
• timestamps:     { createdAt, updatedAt }
```

3. Database Design & Models (Cont.)

Schema 3: Doctor

Stores credentials, qualifications, availabilities, and ratings for clinical practitioners.

• name:	{ Type: String, Required: true }
• email:	{ Type: String, Required: true, Unique: true }
• password:	{ Type: String, Required: true, Select: false }
• specialization:	{ Type: String, Required: true }
• experience:	{ Type: Number, Required: true }
• consultationFee:	{ Type: Number, Required: true }
• hospital:	{ Type: String, Required: true }
• availableSlots:	{ Type: Object, Default: {} }
• profileImage:	{ Type: String, Default: "blank-profile-url" }
• verificationStatus:	{ Type: String, Enum: ["pending", "approved", "rejected"] }
• ratings:	{ Type: Number, Default: 4.5 }
• about:	{ Type: String, Required: true }

Schema 4: Appointment

Links patient documents with doctors. Tracks symptoms, reports and states.

• patientId:	{ Type: Schema.Types.ObjectId, Ref: "User", Required: true }
• doctorId:	{ Type: Schema.Types.ObjectId, Ref: "Doctor", Required: true }
• appointmentDate:	{ Type: String, Required: true }
• appointmentTime:	{ Type: String, Required: true }
• status:	{ Type: String, Enum: ["pending", "confirmed", "cancelled", "completed"] }
• paymentStatus:	{ Type: String, Enum: ["pending", "paid"], Default: "pending" }
• symptoms:	{ Type: String, Required: true }
• reports:	{ Type: [String], Default: [] }
• timestamps:	{ createdAt, updatedAt }

4. Authentication & Security Flow

Security is integrated into all architectural levels, matching industry standards for protecting patient medical history and database configurations.

1. Bcrypt Password Hashing

Before writing documents for Admins, Users, or Doctors to MongoDB, Mongoose middleware hooks catch the password field, salt it (10 rounds), and replace it with a one-way hashed string. Authenticators use a custom Mongoose method schema: `comparePassword()` to safely verify passwords.

2. JSON Web Token (JWT) Generation

On validation of credentials, the backend compiles a token using `jwt.sign()` containing the account's ID. The token expires in 30 days. Client requests include this Bearer string inside the Authorization header.

3. Protected Routes & Middleware Gateway

The `authMiddleware` intercepts routes, checks for "Bearer <token>", verifies it using the `JWT_SECRET`, and retrieves the account across User, Doctor, or Admin collections. Once authenticated, details are appended to `req.user`, allowing role restriction validation (`protect` and `restrictTo`).

```
// Role protection middleware example
router.put("/toggle-availability", protect, restrictTo("doctor"), toggleAvailability);
router.post("/add-doctor", protect, restrictTo("admin"), upload.single("image"), addDoctor);
router.post("/complete", protect, restrictTo("doctor"), completeAppointment);
```

5. Frontend Layout & State Architecture

The React client application is engineered to load views fast, manage global state triggers easily, and update UI layouts dynamically based on device screens.

1. Modern State Management via Context API

- **AuthContext:** Provides global user session data, register controls, login callbacks, profile loading, and profile picture modifications.
- **AppContext:** Coordinates the master doctor listings, appointment queues, online payment mock checkouts, cancellations, and light/dark theme selectors.

2. Axios Client Interceptors

Centralized inside `src/api/axios.js`, an Axios client interceptor catches outgoing requests and injects the stored JWT token. This eliminates the need to attach authorization headers manually.

3. Dynamic Theme System

A Tailwind theme switcher applies classes directly to the root element. Styles automatically switch between medical blue/slate-50 themes and slate-900/slate-950 dark layouts.

4. Responsive Page Layouts

Built utilizing CSS grids, flexboxes, and mobile drawer components. Reusable widgets like `DoctorCard`, `SidebarFilters`, and `AppointmentQueue` adjust between phones and large screens.

6. Local Development Setup & Checklist

Follow these steps to deploy and run the application locally in development mode.

Step-by-Step Instructions

1. Verify Node.js (v18+) and MongoDB are installed locally.
2. Navigate to project files folders/backend directory, create a .env file and set: PORT, MONGODB_URI, JWT_SECRET, and optionally Cloudinary credentials.
3. Run: npm install inside backend directory to download server packages.
4. Start backend using: npm run dev to connect and seed doctor profiles.
5. Navigate to project files folders/frontend directory, verify configurations.
6. Run: npm install to configure React dependencies.
7. Start React server: npm run dev to run client on http://localhost:3000.

Seeded Profiles for Manual Auditing

The seeder sets up the following credentials:

- **Clinician:** richard@bookadoctor.com | password123 (General Physician)
- **Clinician:** emily@bookadoctor.com | password123 (Gynecologist)
- **Administrator:** admin@bookadoctor.com| adminpassword123
- **Patient:** Register custom credentials from the Login page.

This ends the system design documentation.
